

React Patterns & Best Practices

Easy Guide

Contents

1	Introducing Compound Components	2
1.1	What are Compound Components?	2
1.1.1	Real-World Example	2
1.2	Compound Component Pattern	2
1.3	Creating Compound Components	3
1.4	Using Compound Components	4
2	Managing Multi-Component State with Context API	5
2.1	The Problem	5
2.2	Solution: Context API	5
2.3	Context Flow	6
3	Grouping Compound Components	7
3.1	Logical Grouping	7
3.2	File Organization	7
3.3	Exporting Pattern	8
4	Adding Extra Components For Reusability	9
4.1	Extending with Helper Components	9
4.2	Implementation	9
4.3	Benefits	10
5	Sharing Cross-Component State	11
5.1	Context for Shared State	11
5.2	Using Shared State Form	12
5.3	Complex Shared State	12
6	Introducing & Using Render Props	13
6.1	What are Render Props?	13
6.2	Basic Example	13
6.3	Common Pattern	13
7	Adding Search Functionality	15
7.1	Search Component Structure	15
7.2	Search with Filtering	15
7.3	Usage Example	16

8	Implementing Search with Render Props	17
8.1	Advanced Search Component	17
8.2	With Render Props	17
8.3	Search with Highlighting	18
9	Handling Keys Dynamically	20
9.1	Why Keys Matter	20
9.2	Dynamic Key Generation	20
9.3	Composite Keys	20
9.4	Stable Key Pattern	21
10	Working with Debouncing	22
10.1	What is Debouncing?	22
10.2	Custom Debounce Hook	22
10.3	Using Debounce	22
10.4	Debounce with API Call	23
10.5	Performance Benefit	23

Chapter 1

Introducing Compound Components

1.1 What are Compound Components?

Components that work together as a set. Think of them like HTML elements that depend on each other.

1.1.1 Real-World Example

```
<select>
  <option>Option 1</option>
  <option>Option 2</option>
</select>
```

`<select>` and `<option>` work together. Neither makes sense alone.

1.2 Compound Component Pattern

```
// These components work together
<Accordion>
  <Accordion.Item>
    <Accordion.Header>Title</Accordion.Header>
    <Accordion.Content>Content</Accordion.Content>
  </Accordion.Item>
</Accordion>
```

Each component has a role:

- `Accordion` - Container, manages state
- `Accordion.Item` - Each section
- `Accordion.Header` - Title
- `Accordion.Content` - Body

1.3 Creating Compound Components

```
// Accordion.js
import { createContext } from 'react';

const AccordionContext = createContext();

export function Accordion({ children }) {
  const [openId, setOpenId] = React.useState(null);

  return (
    <AccordionContext.Provider value={{ openId, setOpenId }}>
      <div>{children}</div>
    </AccordionContext.Provider>
  );
}

// Item component
function Item({ id, children }) {
  return <div className="accordion-item">{children}</div>;
}

// Header component
function Header({ id, children }) {
  const { openId, setOpenId } = React.useContext(AccordionContext);

  return (
    <button
      onClick={() => setOpenId(openId === id ? null : id)}
      className="accordion-header"
    >
      {children}
    </button>
  );
}

// Content component
function Content({ id, children }) {
  const { openId } = React.useContext(AccordionContext);

  if (openId !== id) return null;

  return <div className="accordion-content">{children}</div>;
}

// Attach sub-components
Accordion.Item = Item;
Accordion.Header = Header;
```

```
Accordion.Content = Content;  
  
export default Accordion;
```

1.4 Using Compound Components

```
import Accordion from './Accordion';  
  
export default function App() {  
  return (  
    <Accordion>  
      <Accordion.Item id="1">  
        <Accordion.Header>Section 1</Accordion.Header>  
        <Accordion.Content>Content 1</Accordion.Content>  
      </Accordion.Item>  
  
      <Accordion.Item id="2">  
        <Accordion.Header>Section 2</Accordion.Header>  
        <Accordion.Content>Content 2</Accordion.Content>  
      </Accordion.Item>  
    </Accordion>  
  );  
}
```

Benefits: Clean API, flexible structure, reusable pattern!

Chapter 2

Managing Multi-Component State with Context API

2.1 The Problem

Multiple child components need to access same state. Props get messy.

```
// Bad: Prop drilling
<Accordion
  openId={openId}
  setOpenId={setOpenId}
>
  <Item openId={openId} setOpenId={setOpenId}>
    <Header openId={openId} setOpenId={setOpenId}>
      ...
    </Header>
  </Item>
</Accordion>
```

Too many props!

2.2 Solution: Context API

```
import { createContext, useContext } from 'react';

// 1. Create context
const AccordionContext = createContext();

// 2. Provide context
export function Accordion({ children }) {
  const [openId, setOpenId] = useState(null);

  return (
    <AccordionContext.Provider value={{ openId, setOpenId }}>
      {children}
    </AccordionContext.Provider>
  );
}
```

```
    </AccordionContext.Provider>
  );
}

// 3. Use context in child components
function Header({ id, children }) {
  const { openId, setOpenId } = useContext(AccordionContext);

  return (
    <button onClick={() => setOpenId(openId === id ? null : id)}>
      {children}
    </button>
  );
}
```

2.3 Context Flow

```
Accordion (provides context)
  Item
    Header (uses context)
    Content (uses context)
  Item
    Header (uses context)
    Content (uses context)
```

Clean: No prop drilling needed!

Chapter 3

Grouping Compound Components

3.1 Logical Grouping

Group related sub-components together:

```
// Accordion.js
export function Accordion({ children }) {
  // ... provider code
}

Accordion.Item = function Item({ id, children }) { /* ... */ };
Accordion.Header = function Header({ id, children }) { /* ... */ };
Accordion.Content = function Content({ id, children }) { /* ... */ };

// Now imported as one unit
export default Accordion;
```

3.2 File Organization

Option 1: Single file

```
components/
  Accordion/
    Accordion.js (all components together)
```

Option 2: Separate files

```
components/
  Accordion/
    index.js      (exports Accordion)
    Item.js
    Header.js
    Content.js
```

3.3 Exporting Pattern

```
// Accordion/index.js
import Accordion from './Accordion';
import Item from './Item';
import Header from './Header';
import Content from './Content';

Accordion.Item = Item;
Accordion.Header = Header;
Accordion.Content = Content;

export default Accordion;
```

Organization: Keep related components together!

Chapter 4

Adding Extra Components For Reusability

4.1 Extending with Helper Components

Add more sub-components for flexibility:

```
<Tabs>
  <Tabs.List>
    <Tabs.Trigger>Tab 1</Tabs.Trigger>
    <Tabs.Trigger>Tab 2</Tabs.Trigger>
  </Tabs.List>

  <Tabs.Content>Content 1</Tabs.Content>
  <Tabs.Content>Content 2</Tabs.Content>
</Tabs>
```

4.2 Implementation

```
export function Tabs({ children }) {
  const [active, setActive] = useState(0);

  return (
    <TabsContext.Provider value={{ active, setActive }}>
      <div className="tabs">{children}</div>
    </TabsContext.Provider>
  );
}

Tabs.List = function List({ children }) {
  return <div className="tabs-list">{children}</div>;
};

Tabs.Trigger = function Trigger({ index, children }) {
  const { active, setActive } = useContext(TabsContext);
```

```
return (  
  <button  
    className={active === index ? 'active' : ''}  
    onClick={() => setActive(index)}  
  >  
    {children}  
  </button>  
)  
);  
};  
  
Tabs.Content = function Content({ index, children }) {  
  const { active } = useContext(TabsContext);  
  
  if (active !== index) return null;  
  return <div className="tabs-content">{children}</div>;  
};
```

4.3 Benefits

- More flexibility
- Cleaner markup
- Easier to extend
- Better separation of concerns

Modularity: Each component does one thing well!

Chapter 5

Sharing Cross-Component State

5.1 Context for Shared State

Multiple component types share state:

```
const FormContext = createContext();

export function Form({ children, onSubmit }) {
  const [values, setValues] = useState({});
  const [errors, setErrors] = useState({});

  return (
    <FormContext.Provider value={{ values, setValues, errors, setErrors, onSubmit }}>
      <form onSubmit={(e) => {
        e.preventDefault();
        onSubmit(values);
      }}>
        {children}
      </form>
    </FormContext.Provider>
  );
}

// Sub-components access context
Form.Input = function Input({ name, label }) {
  const { values, setValues } = useContext(FormContext);

  return (
    <input
      value={values[name] || ''}
      onChange={(e) => setValues({ ...values, [name]: e.target.value })}
      placeholder={label}
    />
  );
};
```

```
Form.Button = function Button({ children }) {  
  return <button type="submit">{children}</button>;  
};
```

5.2 Using Shared State Form

```
<Form onSubmit={((values) => console.log(values))}>  
  <Form.Input name="email" label="Email" />  
  <Form.Input name="password" label="Password" />  
  <Form.Button>Submit</Form.Button>  
</Form>
```

5.3 Complex Shared State

```
const ContextValue = {  
  values: { email: '', password: '' },  
  setValues: (updater) => { /* ... */ },  
  errors: { email: '', password: '' },  
  setErrors: (updater) => { /* ... */ },  
  touched: { email: false, password: false },  
  setTouched: (updater) => { /* ... */ },  
  handleSubmit: (handler) => { /* ... */ }  
};
```

All sub-components can access any state!

Powerful: Shared state across compound components!

Chapter 6

Introducing & Using Render Props

6.1 What are Render Props?

Pass a function as a prop. Receive data from parent.

6.2 Basic Example

```
// Parent provides data via function
<Mouse>
  ({ { x, y } }) => (
    <p>Mouse position: {x}, {y}</p>
  )
</Mouse>

// Mouse component
function Mouse({ children }) {
  const [position, setPosition] = useState({ x: 0, y: 0 });

  useEffect(() => {
    const handleMouseMove = (e) => {
      setPosition({ x: e.clientX, y: e.clientY });
    };

    window.addEventListener('mousemove', handleMouseMove);
    return () => window.removeEventListener('mousemove', handleMouseMove);
  }, []);

  return children(position);
}
```

6.3 Common Pattern

```
// Data fetching with render props
<DataFetcher url="/api/users">
  ({ { data, loading, error } }) => (
```

```
<>
  {loading && <p>Loading...</p>}
  {error && <p>Error: {error}</p>}
  {data && <UserList users={data} />}
</>
  )}
</DataFetcher>

// Implementation
function DataFetcher({ url, children }) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch(url)
      .then(r => r.json())
      .then(d => {
        setData(d);
        setLoading(false);
      })
      .catch(e => {
        setError(e);
        setLoading(false);
      });
  }, [url]);

  return children({ data, loading, error });
}
```

Flexibility: Render props let parent control rendering!

Chapter 7

Adding Search Functionality

7.1 Search Component Structure

```
// Basic search
<Search>
  ({ query, setQuery, results }) => (
    <>
      <input
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Search..."
      />
      <ul>
        {results.map(item => (
          <li key={item.id}>{item.name}</li>
        ))}
      </ul>
    </>
  )}
</Search>
```

7.2 Search with Filtering

```
function Search({ items, children }) {
  const [query, setQuery] = useState('');

  // Filter items based on query
  const results = items.filter(item =>
    item.name.toLowerCase().includes(query.toLowerCase())
  );

  return children({ query, setQuery, results });
}
```

7.3 Usage Example

```
const users = [
  { id: 1, name: 'Alice' },
  { id: 2, name: 'Bob' },
  { id: 3, name: 'Charlie' }
];

<Search items={users}>
  ({ query, setQuery, results }) => (
    <div>
      <input
        value={query}
        onChange={(e) => setQuery(e.target.value)}
      />
      <p>Found {results.length} results</p>
      {results.map(user => (
        <div key={user.id}>{user.name}</div>
      ))}
    </div>
  )
</Search>
```

Pattern: Render props work great for search!

Chapter 8

Implementing Search with Render Props

8.1 Advanced Search Component

```
function SearchableList({ items, searchKey = 'name' }) {
  const [query, setQuery] = useState('');

  const filteredItems = items.filter(item =>
    item[searchKey]
      .toLowerCase()
      .includes(query.toLowerCase())
  );

  const highlightMatch = (text) => {
    const parts = text.split(new RegExp(`\\(${query}\\)` , 'gi'));
    return parts.map((part, i) =>
      part.toLowerCase() === query.toLowerCase()
        ? <mark key={i}>{part}</mark>
        : part
    );
  };

  return {
    query,
    setQuery,
    results: filteredItems,
    highlightMatch,
    count: filteredItems.length
  };
}
```

8.2 With Render Props

```
function Search({ items, render }) {
```

```

const [query, setQuery] = useState('');

const results = items.filter(item =>
  item.name.toLowerCase().includes(query.toLowerCase())
);

return render({
  query,
  setQuery,
  results,
  hasResults: results.length > 0,
  isEmpty: results.length === 0
});
}

// Usage
<Search items={users} render={({ query, setQuery, results, isEmpty }) => (
  <>
    <input value={query} onChange={(e) => setQuery(e.target.value)} />
    {isEmpty && query && <p>No results found</p> }
    {results.map(user => (
      <div key={user.id}>{user.name}</div>
    ))}
  </>
)} />

```

8.3 Search with Highlighting

```

function HighlightedSearch({ items, searchKey, render }) {
  const [query, setQuery] = useState('');

  const results = items.filter(item =>
    item[searchKey].toLowerCase().includes(query.toLowerCase())
  );

  const highlight = (text) => {
    if (!query) return text;

    const regex = new RegExp(`\\(${query}\\)`, 'gi');
    const parts = text.split(regex);

    return parts.map((part, i) =>
      regex.test(part)
        ? <mark key={i}>{part}</mark>
        : part
    );
  };
}

```

```
return render({ query, setQuery, results, highlight });
}
```

Customizable: Render props give full control!

Chapter 9

Handling Keys Dynamically

9.1 Why Keys Matter

Keys help React identify which items have changed:

```
// Bad: Using index as key (unreliable)
{items.map((item, index) => (
  <Item key={index}>{item.name}</Item>
))}

// Good: Using unique identifier
{items.map(item => (
  <Item key={item.id}>{item.name}</Item>
))}
```

9.2 Dynamic Key Generation

When items don't have unique IDs:

```
// Generate unique key from content
function generateKey(item) {
  return `-${item.category}-${item.name}`;
}

{items.map(item => (
  <Item key={generateKey(item)}>{item.name}</Item>
))}
```

9.3 Composite Keys

Combine multiple properties:

```
function CompositeKey({ userId, postId, commentId }) {
  return `user-${userId}-post-${postId}-comment-${commentId}`;
}
```

```
{comments.map(comment => (  
  <Comment  
    key={CompositeKey({ userId, postId, commentId: comment.id })}  
    comment={comment}  
  />  
))}
```

9.4 Stable Key Pattern

```
// Create stable key outside render  
const itemKeys = useMemo(() => {  
  return items.reduce((acc, item) => {  
    acc[item.id] = `item-${item.id}-${item.version}`;  
    return acc;  
  }, {});  
}, [items]);  
  
{items.map(item => (  
  <Item key={itemKeys[item.id]}>{item.name}</Item>  
))}
```

Important: Always use stable, unique keys!

Chapter 10

Working with Debouncing

10.1 What is Debouncing?

Delay function execution until user stops typing.

10.2 Custom Debounce Hook

```
function useDebounce(value, delay = 500) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    // Set timer
    const timer = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    // Cleanup on change
    return () => clearTimeout(timer);
  }, [value, delay]);

  return debouncedValue;
}
```

10.3 Using Debounce

```
function SearchUsers() {
  const [query, setQuery] = useState('');
  const debouncedQuery = useDebounce(query, 300);

  // Only search when user stops typing for 300ms
  useEffect(() => {
    if (debouncedQuery) {
      searchUsers(debouncedQuery);
    }
  }, [debouncedQuery]);
}
```



```
return (  
  <input  
    value={query}  
    onChange={(e) => setQuery(e.target.value)}  
    placeholder="Search..."  
  />  
);  
}
```

10.4 Debounce with API Call

```
function SearchWithAPI() {  
  const [query, setQuery] = useState('');  
  const debouncedQuery = useDebounce(query, 500);  
  const [results, setResults] = useState([]);  
  
  useEffect(() => {  
    if (!debouncedQuery) {  
      setResults([]);  
      return;  
    }  
  
    // Only called 500ms after user stops typing  
    fetch(`/api/search?q=\${debouncedQuery}\`)  
      .then(r => r.json())  
      .then(data => setResults(data));  
  }, [debouncedQuery]);  
  
  return (  
    <div>  
      <input  
        value={query}  
        onChange={(e) => setQuery(e.target.value)}  
      />  
      {results.map(item => (  
        <div key={item.id}>{item.name}</div>  
      ))}  
    </div>  
  );  
}
```

10.5 Performance Benefit

```
// Without debounce  
User types "react"  
- After 'r': API call
```

- After 'e': API call
- After 'a': API call
- After 'c': API call
- After 't': API call

Total: 5 API calls

// With debounce (300ms)

User types "react"

- Waits 300ms after last keypress
- API call happens once

Total: 1 API call

Optimization: Reduce API calls dramatically!

Quick Summary

Key Patterns

1. **Compound Components** - Components that work together
2. **Context API** - Share state without props
3. **Render Props** - Pass functions as props
4. **Debouncing** - Delay execution

Compound Components Structure

```
<Container>
  <Container.Child1>
    <Container.Child1.Subchild />
  </Container.Child1>
  <Container.Child2>
    Content
  </Container.Child2>
</Container>
```

Context Pattern

1. Create context
2. Create provider component
3. Use context in child components
4. Avoid prop drilling

Render Props Pattern

1. Pass function as prop
2. Function receives data
3. Return JSX from function
4. Full control over rendering

Best Practices

1. Use stable, unique keys
2. Debounce search and expensive operations
3. Group related compounds logically
4. Use context for cross-cutting concerns
5. Keep render props clean and simple
6. Don't over-engineer - use when needed

Common Mistakes to Avoid

- Using index as key
- Creating context inside component (recreates on render)
- Unnecessary render props (could use props)
- Compound components too tightly coupled
- No debouncing on frequent events
- Context for everything (use when truly needed)

When to Use Each Pattern

Pattern	When to Use
Compound	Multiple related components
Context	Share state across components
Render Props	Give parent rendering control
Debouncing	Frequent events (search, resize)

Performance Optimization Checklist

- Use unique, stable keys
- Debounce search inputs
- Memoize expensive computations
- Lazy load components
- Use React.memo for expensive renders
- Monitor performance in DevTools

Mastering patterns makes React code more maintainable and performant!